

Enterprise Pattern: Repository



Kevin Dockx

Architect

@KevinDockx | www.kevindockx.com



Coming Up



Detailing the repository pattern

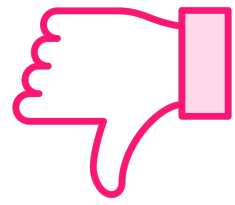
Generic vs non-generic repositories

EF Core and the repository pattern

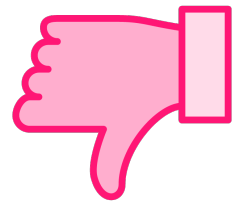
Real-world use cases



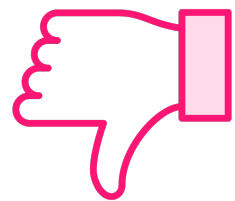
Potential Issues Without a Repository



Code duplication



Error-prone code



Less testable code



Repository pattern

This pattern provides a way to abstract and encapsulate your data access logic from the rest of your application. It aims to make the code, safe for the repository implementation, persistence ignorant.



More important than being able to change persistence technologies is the ability to choose the best-fitting persistence logic for each repository (method)



Detailing the Repository Pattern



Maintainability



Testability



Adaptability



Flexibility



```
public interface IRepository<T>

public class OrderRepository: IRepository<Order>

public class ConsumingClass(IRepository<Order> orderRepository)
{
    // delegate data access tasks to the repository
}
```

Detailing the Repository Pattern



```
public interface IRepository<T>

public class OrderRepository: IRepository<Order>

public class ConsumingClass(IRepository<Order> orderRepository)
{
    // delegate data access tasks to the repository
}
```

Detailing the Repository Pattern

Repository interface



```
public interface IRepository<T>

public class OrderRepository: IRepository<Order>

public class ConsumingClass(IRepository<Order> orderRepository)
{
    // delegate data access tasks to the repository
}
```

Detailing the Repository Pattern

Repository interface

Repository implementation



```
public interface IRepository<T>

public class OrderRepository: IRepository<Order>

public class ConsumingClass(IRepository<Order> orderRepository)
{
    // delegate data access tasks to the repository
}
```

Detailing the Repository Pattern

Repository interface

Repository implementation

Consuming class delegates data access to the repository, often used DI to get an instance of the required repository



Generic Vs Non-generic Repositories

```
public interface IRepository<T>
{
    Task<T?> Get(int id);
    Task<IEnumerable<T>> GetAll();
    void Add(T entity);
    void Delete(T entity);
    void Update(T entity);
    Task SaveChanges();
}
```



Generic Vs Non-generic Repositories

```
public class GenericOrderRepository(OrderDbContext dbContext) :
    IRepository<Order>
{
    private readonly OrderDbContext _dbContext = dbContext;

    public async Task<Order?> Get(int id)
    {
        return await _dbContext.Orders.FindAsync(id);
    }

    public async Task<IEnumerable<Order>> GetAll()
    {
        return await _dbContext.Orders.ToListAsync();
    }

    // ... other method implementations ...
}
```



Advantages of a Generic Repository



Reusability



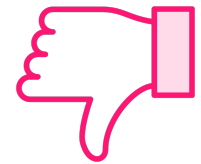
Reduced repetitiveness



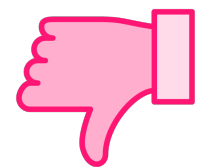
Consistent API



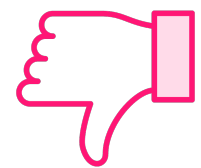
Disadvantages of a Generic Repository



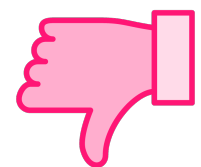
Lack of specificity



Overgeneralization



Performance concerns



Maintenance concerns



Generic Vs Non-generic Repositories

```
public interface IOrderLineRepository
{
    Task<OrderLine?> Get(int id);
    Task<IEnumerable<OrderLine>> GetAll();
    Task<IEnumerable<OrderLine>> GetAllByOrderId(int orderId);
    void Add(OrderLine entity);
    void Delete(OrderLine entity);
    void Update(OrderLine entity);
    Task SaveChanges();
}
```



Generic Vs Non-generic Repositories

```
public interface IOrderLineRepository
{
    Task<OrderLine?> Get(int id);
    Task<IEnumerable<OrderLine>> GetAll();
    Task<IEnumerable<OrderLine>> GetAllByOrderId(int orderId);
    void Add(OrderLine entity);
    void Delete(OrderLine entity);
    void Update(OrderLine entity);
    Task SaveChanges();
}
```



Advantages of a Non-generic Repository



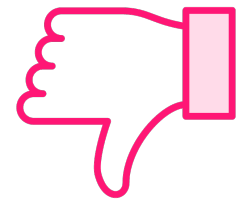
Allows tailored methods



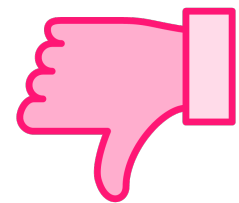
Clear intent



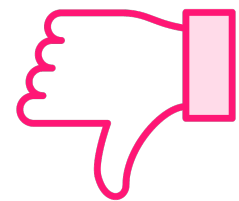
Disadvantages of a Non-generic Repository



Code duplication across interfaces



Greater learning curve



Maintenance concerns



Generic Vs Non-generic Repositories

Don't be afraid to mix and match

- Generic for common CRUD operations
- Non-generic for entity-specific methods



Demo



**Adding a generic repository contract and
non-generic implementation**



Demo



**Adding a generic repository
implementation**



Demo



Adding a non-generic repository contract and implementation



Demo



Combining generic and non-generic repositories



**EF Core provides an
implementation of the
repository pattern**



EF Core and the Repository Pattern

- An EF Core `DbSet<T>` enables interacting with the underlying data store via queries
- It represents a collection for a given entity



The DbSet<T> is a generic repository implementation



EF Core and the Repository Pattern

EF Core isn't very strict in its repository implementation

- `DbSet<T>` implements `IQueryable<T>`, which allows manipulating the database query
- It's considered a leaky abstraction

Mocking used to be difficult



EF Core and the Repository Pattern

That doesn't mean you should avoid it
altogether

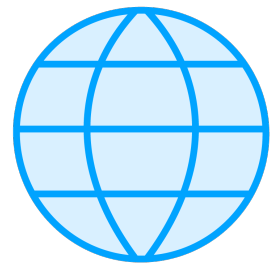
- If your use case asks for it, use it



Software design decisions are often trade-offs. That doesn't make one better than the other by design.



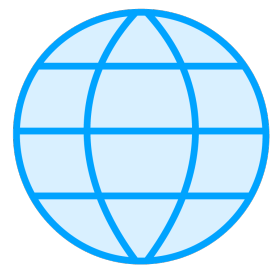
Real-world Use Cases



Library management



E-commerce platform



... and almost all applications that deal with data access



Summary



Purpose of the repository pattern

- Abstract and encapsulate data access logic from the rest of our application

Generic and non-generic implementations are common

- Don't be afraid to combine them



Summary



EF Core provides a (somewhat leaky) implementation via `DbSet<T>`



Up Next:

Enterprise Pattern: Unit of Work

